Secuencias

Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de Río Negro

09 / 2026

r33

Hoy vamos a ver

El tamaño en memoria de las cosas
Arreglos y cadenas
- definición
- declaración
- inicialización
- indexación
- uso como L-Value y R-value
- identidad

Arreglos de largo variable (que no hay que usar)

Operador sizeof

UNRN Universidad Nacional de Río Negro

Operador `sizeof`

`size_t sizeof(variable)`

nos da el tamaño en bytes de la variable o tipo

Y es resuelto por el compilador

Valor tipo `size_t`

Un valor entero sin signo de por lo menos 16-bits que representa un tamaño en memoria

Definido en `stddef.h` e incluido por `stdlib.h`

Funciona con tipos y variables

```
#include <stdio.h>

int main()
{
  int variable;
  printf("Tamaño de char: %zu bytes", sizeof(char));
  printf("Tamaño de int: %zu bytes", variable);
}
```

¡A la terminal!



Arreglos

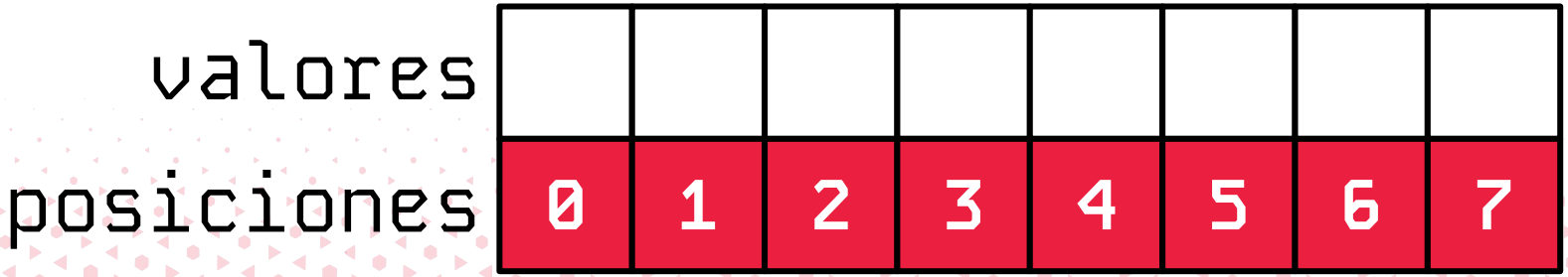
UNRN Universidad Nacional de Río Negro

¿Qué son?

UNRN Universidad Nacional de Río Negro

Memoria contigua

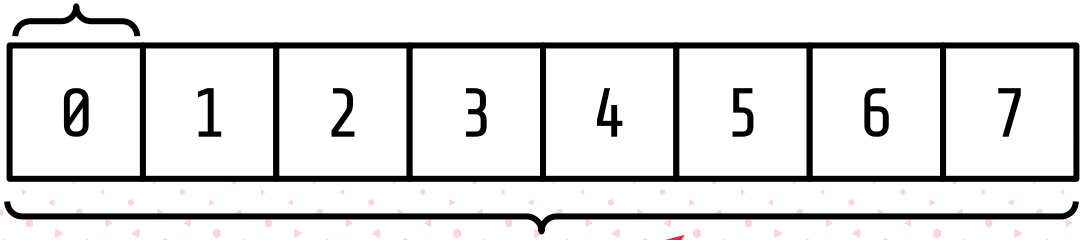
```
int arreglo[8];
```



Memoria contigua

sizeof(int)

```
int arreglo[8];
```



sizeof(arreglo)

Extrayendo la cantidad de elementos de un arreglo

```
int arreglo[10];
size_t largo = sizeof(arreglo) / sizeof(arreglo[0])
```



Declaración

UNRN Universidad Nacional de Río Negro

Declaración

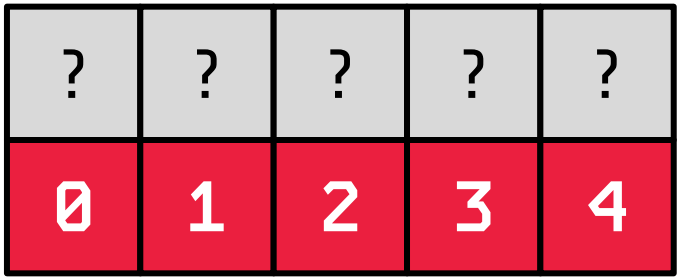



```
tipo identificador[longitud];
```

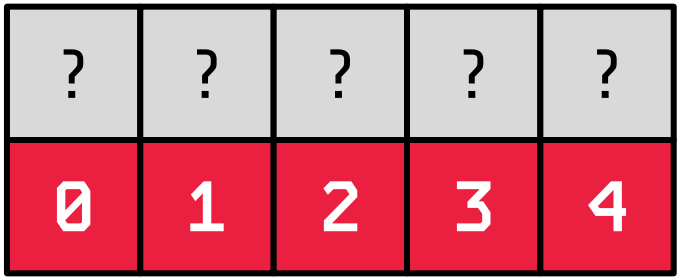
Inicialización

UNRN Universidad Nacional de Río Negro

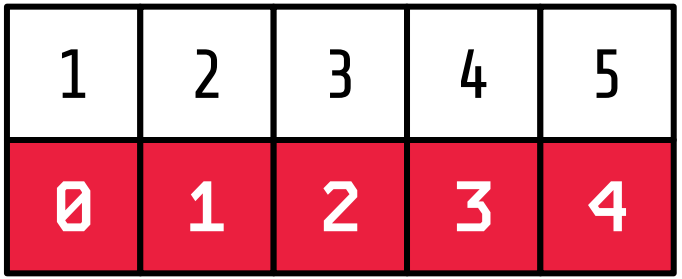
```
int arreglo[5];
```



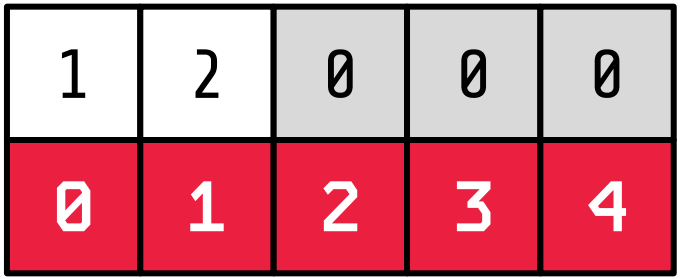
```
int vacia[5] = {0};
```



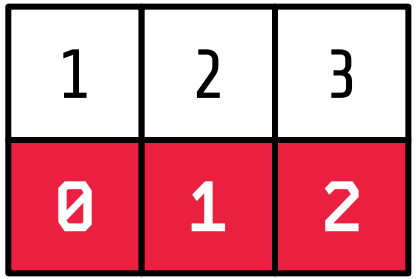
```
int completa[5] = {1, 2, 3, 4, 5};
```



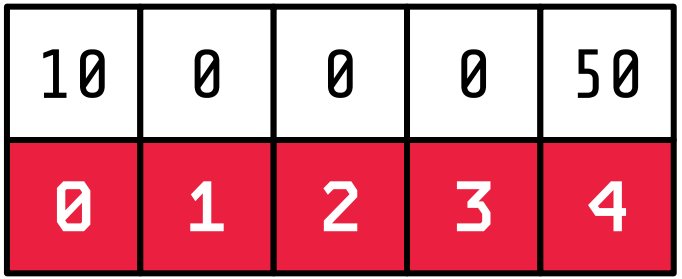
```
int parcial[5] = {1, 2};
```



```
int implicita[] = {1, 2, 3};
```



```
int designada[5] = {[0] = 10, [4] = 50};
```



El operador de indexado

UNRN Universidad Nacional de Río Negro

Recuerden que los índices arrancan en 0

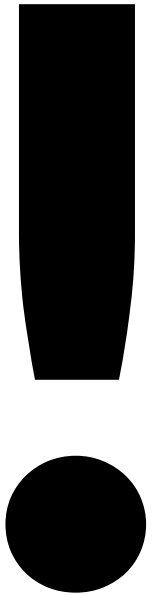
La versión *super* genérica

```
identificador_arreglo[posición]
```

Más adelante vamos a ver qué es *realmente* :-D



No hay verificación de límites



Acceso

Uso como R-Value

Forma genérica

```
L-value = identificador_arreglo[indice];
```

Y cualquier otro uso como R-Value

```
int calificaciones[5] = {10, 8, 9, 7, 10};

// Obtener el valor del primer elemento (índice 0)
int primera = calificaciones[0];
```

Ejemplo con cadenas

```c
#include <stdio.h>

int main()
{
    char antes[] = "XXXXX";
    char cadena[5] = "Hola";
    char despues[] = "YYYYY";
    printf("cadena: %s\n", cadena);
    printf("%c\n", cadena[-4]);
    printf("%c\n", cadena[6]);
    return 0;
}
```

¿Cual es la salida?

Modificaciones

Uso como L-Value

Forma genérica

```
identificador_arreglo[indice] = R-value;
```

Modificaciones en un arreglo

```
int edades[4] = {20, 25, 22, 28};
printf("La edad tres es: %d\n", edades[2]);
edades[2] = 23;
printf("La nueva edad tres es: %d\n", edades[2]);
```

Identidad

Los identificadores, en sí, *son especiales*

```
int arr1[5] = {1, 2, 3, 4, 5};
int arr2[5] = {10, 20, 30, 40, 50};

// La siguiente línea causará un error de compilación.

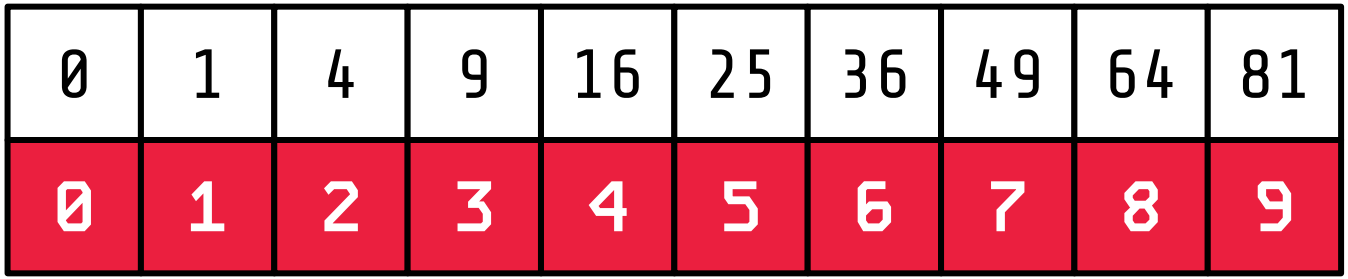
arr1 = arr2; // Error: expression is not assignable.
```

¡Previene fugas de memoria!

Recorrido

Marida muy bien con lazos `for`

```
int arreglo[10] = {};
size_t largo = sizeof(arreglo) / sizeof(arreglo[0]);
for (size_t i = 0; i < largo; i++)
{
  arreglo[i] = i * i;
}
```


0 1 4 9 16 25 36 49 64 81
0 1 2 3 4 5 6 7 8 9

Las variables que refieran al tamaño o posición de un arreglo deben ser `size_t`

Comportamiento no definido

UNRN Universidad Nacional de Río Negro

Comportamiento indefinido (undefined behavior)

El compilador, al encontrar dicha situación, puede generar **cualquier resultado**.

(el estándar no dice qué hacer)

Lectura de arreglos fuera de sus límites

```
int arr[5];
int valor = arr[10];
```

Esto es más divertido con un lazo

Cuando nos pasamos de los límites de un arreglo

```
int arreglo[10];
for (size_t i = 0; i < 100 ; i++)
{
  printf("%zu/%d|",i, arreglo[i]);
}
```

¡A la terminal!



Asignación a arreglos fuera de sus límites

```
int arr[5];
arr[10] = 5;
```

Pueden suceder cosas interesantes

```
int arreglo[10];
for (size_t i = 0; i < 100 ; i++)
{
  arreglo[i] = 0;
  printf("%zu/%d|",i, arreglo[i]);
}
```

¡A la terminal!



{ Otro comportamiento *no definido* }

Leer una variable no inicializada

```
int x;
printf("%d", x);
```

¡Quizás sea cero!


¿Preguntas?

Arreglos de largo variable [ALV]

UNRN Universidad Nacional de Río Negro

La definición que vimos de `arreglo`

```
int arreglo[3];
```


¡Pero con una variable funciona!

Pero, pero peerero, no es correcto

```
int tamanio;
scanf("%d", &tamanio);
int arreglo[tamanio];
```



El tamaño de un arreglo solo puede ser definido al compilar*

*más adelante vamos a ver cómo liberarnos de esta restricción

Ya vamos a ver

por qué no

es correcto

Los arreglos *estáticos* solo pueden ser creados de un tamaño fijo al compilar

Arreglos y funciones

UNRN Universidad Nacional de Río Negro

Declaración prototipo

UNRN Universidad Nacional de Río Negro

Esta parece ser una buena primera función.

```
size_t largo(int arreglo[])
{
    return sizeof(arreglo) / sizeof(arreglo[0]);
}
```

¡A la terminal!



¡El arreglo *decae* en algo diferente!

Qué vamos a ver la proxima clase

{

No se fíen de `sizeof` en arreglos como argumentos de funciones*

}

Por esto....

Para cuando una función recibe un arreglo

```
tipo_retorno funcion(tipo arreglo[], size_t tamanio);
```

podría ser `int`

Es necesario indicar el tamaño


¿Preguntas?

NULL

Palabra reservada que indica que un arreglo no fue inicializado*



Efectos secundarios II

UNRN Universidad Nacional de Río Negro

Para esta función

```
/**
 * Busca el valor máximo en un arreglo de enteros.
 *
 * @param arreglo como la secuencia de números.
 * @param largo   es la cantidad de elementos en arreglo.
 * @return el valor máximo encontrado en el arreglo.
 */
int maximo(int arreglo[], size_t tamanio);
```

Con esta implementación

```
int maximo(int arreglo[], size_t largo)
{
    for (size_t i = 1; i < largo; i++) {
        if (arreglo[i] > arreglo[0]) {
            arreglo[0] = arreglo[i];
        }
    }

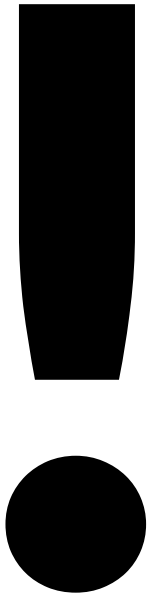
    return arreglo[0];
}
```

¡A la terminal!





¿No que pasa *todo* por copia?

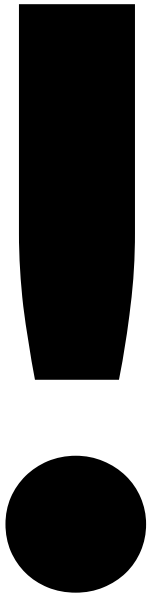


Si... pero

Los arreglos son una ubicación en la memoria



Los cambios en la función *cambian* el arreglo original




¿Preguntas?

constantes

UNRN Universidad Nacional de **Río Negro**

Como calificador en variables

```
const int MAXIMO = 100;
// numero_maximo = 200; // Error: intento de modificar un
valor constante
```

Implica que es necesario inicializarlo en la declaración

Las variables declaradas como `const` van en MAYUSCULAS

Como calificador en argumentos

```
void imprimir_cadena(const char cadena[], size_t capacidad)
{
    printf("Cadena: %s\n", s);
    // s[0] = 'X';
}
```

```
error: assignment of read-only location ‘*cadena’
```

Nos aseguramos de que no se modifique el contenido

Los argumentos de funciones deben ser `const` siempre que sea posible

{ Para asegurarnos de no tener efectos secundarios *accidentales* }


¿Preguntas?

Y que nos lleva a

UNRN Universidad Nacional de Río Negro

Retorno de arreglos en funciones

UNRN Universidad Nacional de Río Negro

Dado este código

```
int[] ordena(int arreglo[], size_t largo)
{
  int retorno[];
  // ordenamo
  return retorno;
}
```

Retornos

Retornar un arreglo creado en una función acarrea **comportamiento no definido**

{ Los retornos de arreglos son una copia de la dirección de memoria }

Esto significa que los arreglos se declaran fuera de las funciones


¿Preguntas?

Documentación

de arreglos

UNRN Universidad Nacional de Río Negro

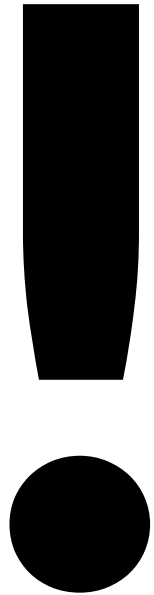
Descripción de la función con arreglos

```
/*
 * Este procedimiento se encarga de ordenar el arreglo
 *   pasado como argumento 'en el lugar'.
 * @param arreglo los valores a ordenar
 *     pre: el arreglo es válido y posee elementos
 *     post: el arreglo es modificado, quedando ordenado
 * @param capacidad es la cantidad de elementos en el arreglo
 *     pre: el valor es consecuente con el arreglo
 */
void ordena(int arreglo[], size_t capacidad);
```

{ **Recuerden que las postcondiciones ‘*generales*’ son correctas también.** }



Es muy necesario documentar que los argumentos pueden cambiar




¿Preguntas?

Cadenas

arreglos de caracteres

UNRN Universidad Nacional de Río Negro

characteres individuales

```
char caracter = 'A';
```

Caracteres ASCII de control

00	NULL	(carácter nulo)
01	SOH	(inicio encabezado)
02	STX	(inicio texto)
03	ETX	(fin de texto)
04	EOT	(fin transmisión)
05	ENQ	(consulta)
06	ACK	(reconocimiento)
07	BEL	(timbre)
08	BS	(retroceso)
09	HT	(tab horizontal)
10	LF	(nueva línea)
11	VT	(tab vertical)
12	FF	(nueva página)
13	CR	(retorno de carro)
14	SO	(desplaza afuera)
15	SI	(desplaza adentro)
16	DLE	(esc.vínculo datos)
17	DC1	(control disp. 1)
18	DC2	(control disp. 2)
19	DC3	(control disp. 3)
20	DC4	(control disp. 4)
21	NAK	(conf. negativa)
22	SYN	(inactividad sínc)
23	ETB	(fin bloque trans)
24	CAN	(cancelar)
25	EM	(fin del medio)
26	SUB	(sustitución)
27	ESC	(escape)
28	FS	(sep. archivos)
29	GS	(sep. grupos)
30	RS	(sep. registros)
31	US	(sep. unidades)
127	DEL	(suprimir)

Caracteres ASCII imprimibles

32	espacio	64	@	96	`
33	!	65	A	97	a
34	"	66	B	98	b
35	#	67	C	99	c
36	$	68	D	100	d
37	%	69	E	101	e
38	&	70	F	102	f
39	'	71	G	103	g
40	(	72	H	104	h
41	)	73	I	105	i
42	*	74	J	106	j
43	+	75	K	107	k
44	,	76	L	108	l
45	-	77	M	109	m
46	.	78	N	110	n
47	/	79	O	111	o
48	0	80	P	112	p
49	1	81	Q	113	q
50	2	82	R	114	r
51	3	83	S	115	s
52	4	84	T	116	t
53	5	85	U	117	u
54	6	86	V	118	v
55	7	87	W	119	w
56	8	88	X	120	x
57	9	89	Y	121	y
58	:	90	Z	122	z
59	;	91	[	123	{
60	<	92	\	124	|
61	=	93	]	125	}
62	>	94	^	126	~
63	?	95	_		

¡Secuencias de escape!

`\n` - salto de línea

`\t` - tabulador

`\\` - la barra invertida misma

`\"` - la comilla doble

`\'` - la comilla simple

Como darle un significado alternativo a los mismos

Cadenas

Una cadena “literal”

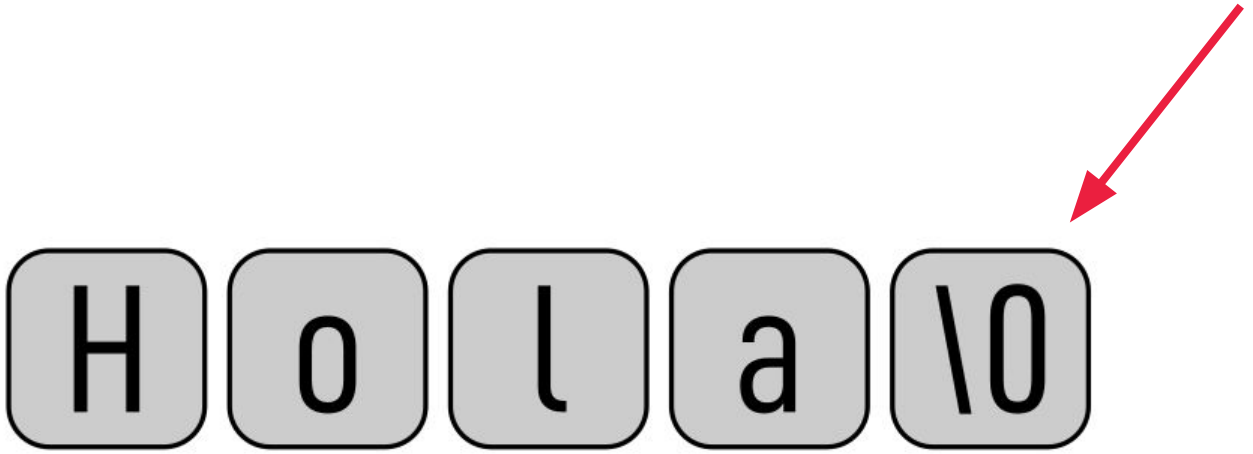


```
char cadena[] = "hola";
```

¿cuántos caracteres tiene “Hola”?

UNRN Universidad Nacional de Río Negro

Las cadenas en C tienen un carácter adicional



Qué indica dónde termina.

Cadenas

indica cuantos `char` guardar



```
char cadena[5] = "hola";
```

Podemos indicar nosotros el tamaño

{ Con la declaración implícita, **va a lo justo y necesario +1** }

Consideraciones

```
char cadena[5] = "hola";
```

Con la **declaración completa**,

¡no se olviden del '\0'!

Largo de una cadena

Caracteres hasta el `\0` en la cadena.

Capacidad de una cadena

Cantidad de espacio reservado para el **arreglo** de caracteres de la cadena.

¿Cuál es el largo y la capacidad de cada variable?

```c
#include <stdio.h>

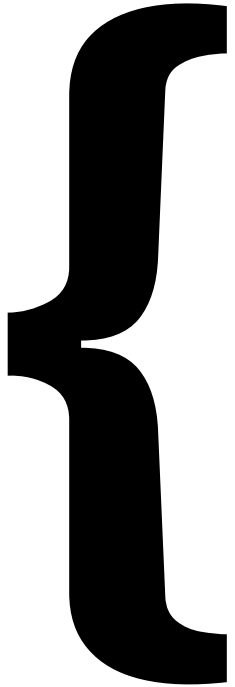
int main()
{
    char primera[] = "Hola Mundo";
    char segunda[6] = "Programacion";
    char tercera[] = "Adios Mundo\n";
    printf("primera: %s\n", primera);
    printf("segunda: %s\n", segunda);
    printf("tercera: %s\n", tercera);
    return 0;
}
```

Como podemos saber

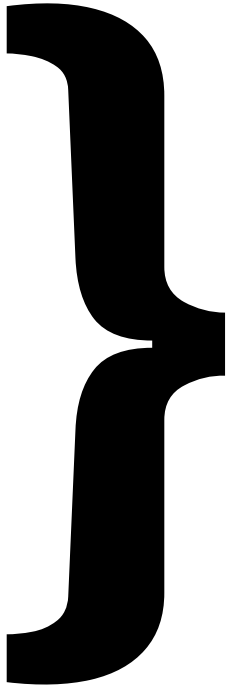
```
#include <string.h>

int strlen ( char[] str );
```

Cuenta los caracteres hasta el `\0`



Cadenas seguras



Como los problemas de los arreglos están presentes en las cadenas

Agregamos el `size_t` de las cadenas

Todas las cadenas en funciones deben ser seguras

Un detalle sobre literales de cadena

UNRN Universidad Nacional de Río Negro

Sobre los literales de cadena

```
void nombre_propio(char cadena[], const size_t capacidad)
{
  char primero = cadena[0];
  cadena[0] = toupper(primero);
  printf("%s\n", cadena);
}
int main() {
  nombre_propio("Martin");
}
```

¿Cual es la salida?


¿Preguntas?

¡A la terminal!



Pantallazo a la `stdlib`

UNRN Universidad Nacional de Río Negro

Manipulación de cadenas

```
size_t strlen( const char* str );
char *strcat( char *dest, const char *src );
char* strcpy( char* dest, const char* src );
int strcmp( const char* lhs, const char* rhs );
```

ctype.h - consultas y manipulación de caracteres

```
int isalnum(int c);
int isalpha(int c);
int isblank(int c);
int iscntrl(int c);
int isdigit(int c);
int isgraph(int c);
int islower(int c);
```

```
int isprint(int c);
int ispunct(int c);
int isspace(int c);
int isupper(int c);
int isxdigit(int c);
int tolower(int c);
int toupper(int c);
```

TP2

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar